



UNIVERSIDADE D
COIMBRA

Filipe de Ornelas Diogo Obrist

TOWARDS THE NEUROEVOLUTION OF TRANSFORMER ARCHITECTURES

Dissertation in the context of the Master Data Science and Engineering,
specialization in Neuroevolution, advised by Professor João Nuno Gonçalves Costa
Cavaleiro Correia and Professor Nuno António Marques Lourenço and presented
to the Department of Informatics Engineering of the Faculty of Sciences and
Technology of the University of Coimbra.

January of 2026



DEPARTAMENTO DE
ENGENHARIA INFORMÁTICA
**FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE D
COIMBRA**

Filipe de Ornelas Diogo Obrist

TOWARDS THE NEUROEVOLUTION OF TRANSFORMER ARCHITECTURES

**Dissertation in the context of the Master Data Science and Engineering,
specialization in Neuroevolution, advised by Professor João Nuno Gonçalves
Costa Cavaleiro Correia and Professor Nuno António Marques Lourenço and
presented to the Department of Informatics Engineering of the Faculty of
Sciences and Technology of the University of Coimbra.**

January of 2026



DEPARTAMENTO DE
ENGENHARIA INFORMÁTICA
**FACULDADE DE
CIÊNCIAS E TECNOLOGIA
UNIVERSIDADE DE
COIMBRA**

Filipe de Ornelas Diogo Obrist

RUMO À NEUROEVOLUÇÃO DE ARQUITETURAS TRANSFORMER

**Dissertação no âmbito do Mestrado em Engenharia e Ciência de Dados,
especialização em Neuroevolução, orientada pelo Professor João Nuno
Gonçalves Costa Cavaleiro Correia e Professor Nuno António Marques
Lourenço e apresentada ao Departamento de Engenharia Informática da
Faculdade de Ciências e Tecnologia da Universidade de Coimbra.**

Janeiro de 2026

STATEMENT ON THE USE OF ARTIFICIAL INTELLIGENCE TOOLS

This document was reviewed and refined with the assistance of Large Language Models, such as ChatGPT, or similar tools, which helped check grammar, correct typos, and enhance clarity. Any Artificial Intelligence-generated text or content is clearly indicated within the document and is used only to a limited extent. The overall content and ideas remain solely the responsibility of the author.

Abstract

Recently, the field of natural language processing has experienced significant advances, driven by the development of large language models predominantly based on Transformer architectures. These models have demonstrated remarkable capabilities in text understanding and generation, but they also present relevant challenges in terms of computational efficiency, scalability, and reliance on global attention mechanisms.

As an alternative, state space models and hybrid architectures that combine different sequence modeling paradigms have emerged, aiming to mitigate some of the limitations inherent to traditional Transformers. In parallel, evolutionary computation has been explored as a means of automating neural architecture design, although its application to modern language models has largely remained restricted to narrow search spaces or higher levels of abstraction.

With this in mind, this work investigates the use of evolutionary computation for the exploration of hybrid architectures that combine attention mechanisms with state space models. The objectives include analysing the potential of this approach to support the discovery of alternative architectural configurations, as well as assessing its feasibility under realistic computational constraints.

This document presents a theoretical background on language models, hybrid architectures, and evolutionary search methods, as well as the practical work developed during the first semester. The proposed approach outlines the initial steps to address the research questions, while the work plan details the tasks planned for the remaining development period. The document concludes with a reflection on the insights obtained during this initial phase and with the definition of the next steps of the research.

Keywords

Neural Architecture Search, Evolutionary Computation, Transformers, Mamba, Hybrid Architectures, Large Language Models.

Resumo

Recentemente, a área do processamento de linguagem natural tem registado avanços significativos, impulsionados pelo desenvolvimento de grandes modelos de linguagem maioritariamente baseados em arquiteturas Transformer. Estes modelos demonstram capacidades notáveis na compreensão e geração de texto, mas apresentam desafios relevantes em termos de eficiência computacional, escalabilidade e dependência de mecanismos de atenção global.

Como alternativa, têm surgido modelos de espaço de estados e arquiteturas híbridas que combinam diferentes paradigmas de modelação de sequências, procurando mitigar algumas das limitações inerentes aos Transformers tradicionais. Paralelamente, a computação evolucionária tem sido explorada como uma forma de automatizar o desenho de arquiteturas neuronais, embora a sua aplicação a modelos de linguagem modernos permaneça limitada a espaços de pesquisa restritos ou a níveis de abstração elevados.

Tendo isto em mente, este trabalho investiga a utilização de computação evolucionária para a exploração de arquiteturas híbridas que combinam mecanismos de atenção e modelos de espaço de estados. Os objetivos incluem analisar o potencial desta abordagem para apoiar a descoberta de configurações arquiteturais alternativas, bem como avaliar a sua viabilidade sob restrições computacionais realistas.

O presente documento apresenta um enquadramento teórico sobre modelos de linguagem, arquiteturas híbridas e métodos de pesquisa evolucionária, assim como o trabalho prático desenvolvido durante o primeiro semestre. A abordagem proposta descreve os passos iniciais para abordar as questões de investigação, enquanto o plano de trabalho delineia as tarefas previstas para o período de desenvolvimento restante. O documento conclui com uma reflexão sobre as intuições obtidas nesta fase inicial e com a definição dos próximos passos da investigação.

Palavras-Chave

Pesquisa de Arquiteturas Neuronais, Computação Evolucionária, Transformers, Mamba, Arquiteturas Híbridas, Grandes Modelos de Linguagem

Contents

1	Introduction	1
1.1	Objectives and Research Questions	3
1.2	Document Structure	3
2	Background	5
2.1	Transformers	5
2.2	State Space Models and Mamba	7
2.3	Hybrid Sequence Modeling Architectures	9
2.4	Large Language Models	11
2.5	Evolutionary Neural Architecture Search	12
2.5.1	Evolutionary Algorithms	13
2.5.2	Evolutionary Algorithms Applied to Neural Architecture Search	14
2.6	State of the Art	15
2.6.1	Evolutionary NAS for Transformer Architectures	16
2.6.2	Evolutionary Optimization of Large Language Models	17
2.6.3	Evolutionary Search Beyond Transformers	18
2.6.4	Summary of Limitations and Research Gaps	19
3	Proposed Methodology	21
4	Preliminary Work	25
5	Work Plan and Timeline	29
6	Conclusion	33
	References	35

List of Figures

2.1	Transformer architecture introduced by Vaswani et al.	6
2.2	Decoder-only Transformer architecture commonly used in large language models.	7
2.3	Canonical State Space Model formulation, illustrating latent state evolution governed by input (B), transition (A), and output (C) matrices.	8
2.4	Overview of the Mamba selective state space model. Input-dependent projections control the discretization and state update dynamics, enabling selective information flow and hardware-efficient computation.	8
2.5	High-level block composition of the Jamba architecture. The model interleaves Transformer layers, Mamba layers, and their Mixture-of-Experts (MoE) variants, forming a heterogeneous chain of processing blocks.	10
2.6	Evolutionary Algorithm Cycle.	13
3.1	High-level overview of the proposed methodology.	22
5.1	Planned work schedule for the second semester of the dissertation.	30
5.2	GitHub issue backlog used to organize and track the remaining work of the dissertation.	30

List of Tables

2.1	Comparison of representative sequence modeling approaches. . . .	20
4.1	Prototype experimental setup.	26

Chapter 1

Introduction

Large Language Models (LLMs) have become a central component of modern Artificial Intelligence, achieving remarkable performance across a wide range of natural language processing tasks, including text generation, translation, summarization, and scientific reasoning. These advances have been driven primarily by the Transformer architecture, whose self-attention mechanism enables effective modeling of long-range dependencies in sequential data [34]. As a consequence, most contemporary LLMs rely on increasingly deep and wide Transformer-based architectures, often comprising hundreds of millions or even billions of parameters [17, 34].

Despite their success, Transformer-based models present some challenges. Their reliance on global self-attention leads to quadratic computational and memory complexity with respect to sequence length, resulting in high training and inference costs [34]. These requirements limit accessibility, increase energy consumption, and raise concerns regarding scalability and sustainability. Furthermore, the design of Transformer architectures remains largely manual, relying on expert intuition and incremental refinements of existing models rather than principled exploration of the architectural design space [2, 34]. This manual process risks converging prematurely on over-engineered or suboptimal solutions, particularly when adapting models to new domains or deployment constraints.

These computational and design limitations have motivated growing interest in automated approaches to architecture discovery. In parallel with architectural scaling, research has explored automated methods for neural architecture design. Evolutionary Neural Architecture Search (ENAS) and related approaches have demonstrated that Evolutionary Computation (EC) can successfully optimize neural network structures, often discovering competitive or novel architectures without explicit human intervention [4, 5, 31, 39]. Within the context of LLMs, however, such approaches have been applied almost exclusively to homogeneous Transformer-based architectures, focusing on variations in depth, attention patterns, or parameter allocation. While promising, this line of work remains constrained to a single architectural paradigm.

More recently, alternative sequence modeling approaches have gained attention as potential complements or alternatives to Transformers. State Space Models

(SSMs), and in particular the Mamba architecture, have emerged as efficient sequence models with linear-time complexity and improved scalability to long contexts [6, 11, 27]. Unlike Transformers, which rely on global attention over all past tokens, Mamba employs selective state mechanisms to retain relevant historical information while maintaining computational efficiency [11, 12]. These properties suggest that the dominance of Transformers is no longer absolute and that alternative modeling paradigms may offer superior trade-offs between expressiveness and efficiency [6, 11].

Building on these ideas, hybrid architectures that combine attention-based and state-space-based components—such as Transformer–Mamba models (e.g., Jamba, Zamba)—have recently been proposed [10, 22]. These models show that fundamentally different sequence modeling mechanisms can coexist within a single network, potentially leveraging the strengths of each. However, current hybrid architectures are still manually designed, relying on expert-driven decisions regarding how and where different components should be integrated. As a result, large portions of the hybrid architectural design space remain unexplored, and it is unclear whether existing designs represent optimal or robust solutions.

Crucially, despite the growing diversity of architectural building blocks, EC has not yet been systematically applied to heterogeneous architecture spaces that include both attention-based and state-space-based components. Existing work on architecture search for Mamba-related models is scarce, recent, and typically limited to highly constrained or training-free settings rather than evolutionary [9]. To date, there has been no comprehensive research of evolutionary methods for discovering or optimizing hybrid Transformer–Mamba architectures. This gap motivates the present work.

Within this broader context, the AMALIA project provides a realistic and relevant application setting. The project aims to develop large-scale language models for European Portuguese, alongside domain-specialized variants, including scientific-domain models developed at the University of Coimbra. While this work does not seek to optimize or deploy production-ready AMALIA models, the tasks relevant to AMALIA—such as text classification, summarization, and domain adaptation—offer practical evaluation scenarios and constraints. These tasks ground the investigation in real-world requirements while allowing systematic exploration of architectural design choices.

Accordingly, this work will explore ENAS as a general mechanism for searching the design space of large language models. Rather than proposing a single fixed architecture, the focus is on assessing whether EC can effectively explore heterogeneous architecture spaces and facilitate the discovery of efficient and competitive hybrid sequence models. By extending evolutionary NAS beyond purely Transformer-based designs to include emerging architectures such as Mamba [11, 9], this work aims to contribute empirical evidence and methodological insights toward the automated design of next-generation LLM architectures.

1.1 Objectives and Research Questions

The main objective of this dissertation is to investigate the feasibility and effectiveness of EC for neural architecture search in hybrid large language model architectures that combine Transformer-based and Mamba-based components. The goal is not to optimize a single production model, but to assess whether evolutionary methods can systematically explore heterogeneous architectural design spaces and identify competitive or efficient architectures under realistic training constraints. The proposed approaches are evaluated on language understanding tasks relevant to large language models, including text classification and related benchmarks.

To structure this investigation, the following research questions are formulated:

- **RQ1 — Can we use EC to evolve heterogeneous Transformer-Mamba architectures?** This question aims to explore the automation of the architectural design of Large Language Models (LLMs) by using evolutionary algorithms to find the optimal arrangement of Attention (Transformer) and State Space Model (Mamba) layers. The goal is to study the viability of finding non-intuitive hybrid topologies that combine the high-quality retrieval capabilities of Transformers with the linear-time inference efficiency of Mamba. To answer this question, we will need to devise a representation and corresponding variation operators to encode hybrid Transformer–Mamba architectures.
- **RQ2 — Can evolutionary search discover hybrid architectures with competitive performance compared to manually designed baselines?** This question is addressed by evaluating evolved architectures on downstream tasks (e.g. classification) and comparing their performance against fixed Transformer-only, Mamba-only, and manually specified hybrid models.

These research questions aim to clarify whether EC can serve as a practical and effective tool for exploring and designing hybrid large language model architectures beyond the Transformer paradigm.

1.2 Document Structure

This document presents the work carried out during the first semester of the dissertation, together with the proposed methodology and the planned objectives for the second semester.

The remainder of this document is organised as follows. Chapter 2 provides the necessary background and theoretical foundations for this work, introducing Transformer architectures, state space models, hybrid sequence modeling approaches, large language models, and EC applied to neural architecture search. It also reviews the state-of-the-art and highlights existing limitations and research gaps.

Chapter 3 presents the proposed methodology of the dissertation, detailing the overall research direction, the hybrid sequence modeling framework, and the role of EC in exploring constrained hybrid architecture spaces.

Chapter 4 describes the work carried out during the first semester, including exploratory implementations of Transformer and Mamba architectures, the development of a hybrid Jamba-based prototype, and a preliminary evolutionary experiment demonstrating the feasibility of the proposed approach.

Chapter 5 outlines the work plan and timeline for the second semester, presenting the remaining research tasks, milestones, risk assessment, and mitigation strategies.

Finally, Chapter 6 concludes the document by summarizing the progress achieved so far and discussing readiness to proceed to the next phase of the dissertation.

Chapter 2

Background

This chapter provides the conceptual and technical foundations necessary to contextualize the research questions addressed in this work. It introduces the principal sequence modeling paradigms underlying modern language models, alongside the evolutionary optimization methods that motivate the proposed approach.

We begin by reviewing Transformer architectures, which form the backbone of most contemporary large language models, and subsequently discuss state space models and recent alternatives designed to address the computational limitations of attention-based mechanisms. Building on these perspectives, hybrid sequence modeling architectures that combine multiple modeling paradigms are introduced. The chapter then outlines the principles of ENAS, establishing the methodological basis for automated architectural exploration. Finally, the state-of-the-art is reviewed to identify existing limitations and research gaps that motivate the evolutionary investigation of hybrid sequence modeling architectures pursued in this work.

2.1 Transformers

Transformers constitute the dominant architectural paradigm in modern natural language processing and large-scale language modeling. Introduced to address the limitations of recurrent architectures—namely inherently sequential computation, limited parallelism, and difficulty in modeling long-range dependencies due to vanishing and exploding gradients—Transformers replace sequential recurrence with a fully parallelizable mechanism based on self-attention [34]. This design enables efficient modeling of long-range dependencies while significantly improving training scalability on modern hardware.

At the core of the Transformer architecture is the self-attention mechanism, which allows each token in a sequence to attend to all other tokens simultaneously [34]. By computing attention weights based on pairwise token interactions, Transformers can dynamically weight contextual information across an entire sequence. This capability is particularly effective for capturing global dependencies and semantic relationships that may span long textual contexts.

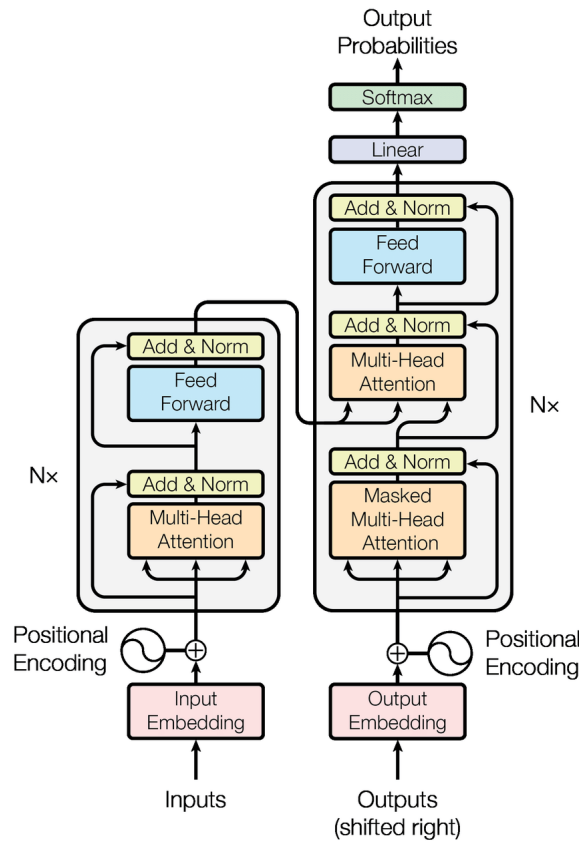


Figure 2.1: Transformer architecture introduced by Vaswani et al.

A standard Transformer layer typically consists of a multi-head self-attention module followed by a position-wise feedforward network, with residual connections and layer normalization applied throughout [34]. Stacking multiple such layers yields deep models capable of learning hierarchical representations of language. Positional information, which is not inherently encoded by attention mechanisms, is incorporated through positional embeddings or encodings.

While the original Transformer architecture was proposed with separate encoder and decoder components (Figure 2.1), most contemporary large language models adopt simplified variants tailored to specific tasks. Encoder-only Transformers are particularly prevalent in representation learning and classification tasks, where bidirectional context is essential [17]. In contrast, generative large language models typically rely on decoder-only autoregressive architectures.

Decoder-only Transformers (Figure 2.2) generate text by predicting the next token conditioned on all previous tokens, using causal masking to preserve the autoregressive property. Notably, the base language model of the AMALIA project, EuroLLM [26], follows this decoder-only Transformer design, consistent with its focus on multilingual text generation.

Despite their success, Transformers exhibit several well-known limitations. The computational and memory complexity of self-attention scales quadratically with sequence length, making long-context processing expensive [2, 34]. Furthermore, the lack of an explicit inductive bias for sequential or temporal structure means that Transformers rely heavily on data and scale to learn such patterns implicitly.

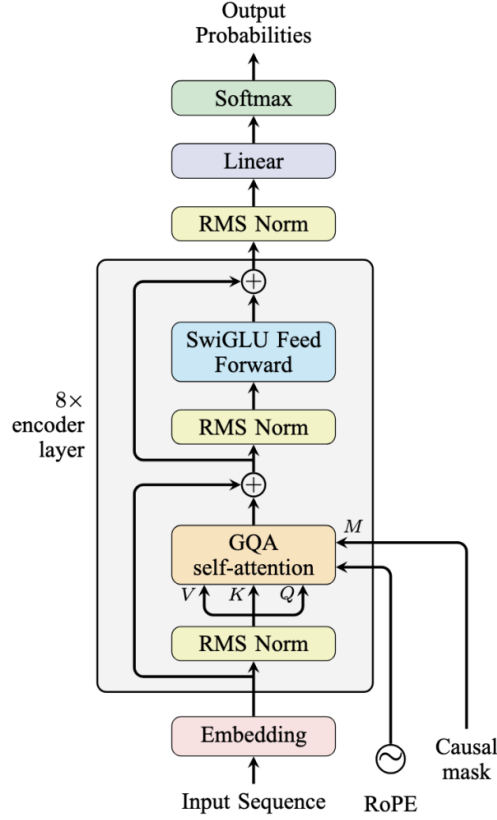


Figure 2.2: Decoder-only Transformer architecture commonly used in large language models.

These limitations have motivated the exploration of alternative sequence modeling paradigms, including State Space Models, discussed in the following section.

2.2 State Space Models and Mamba

State Space Models (SSMs) represent a fundamentally different approach to sequence modeling, grounded in dynamical systems theory [6, 27]. Unlike Transformers, which rely on pairwise token interactions, SSMs model sequences through latent state evolution governed by continuous or discrete-time dynamics. At each time step, the model updates an internal state based on the current input and previous state, producing an output that reflects accumulated historical information [27].

Figure 2.3 illustrates the canonical formulation of a state space model. The input sequence x_t is first projected into the state space through an input matrix B , after which the latent state h_t evolves according to a linear transition defined by matrix A . The output y_t is then generated by projecting the state through matrix C , optionally combined with a direct input-to-output connection via matrix D . This formulation highlights the core idea of SSMs: sequence modeling is achieved by repeatedly updating a hidden state that serves as a structured memory of past inputs.

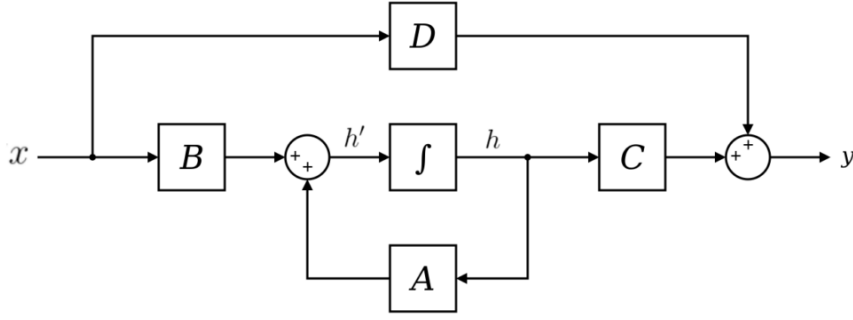


Figure 2.3: Canonical State Space Model formulation, illustrating latent state evolution governed by input (B), transition (A), and output (C) matrices.

Historically, Recurrent Neural Networks (RNNs) can be viewed as a class of learned discrete-time state space models [6]. While RNNs introduced an explicit inductive bias for sequential data, they suffered from severe training challenges, including vanishing and exploding gradients, limited parallelism, and difficulty modeling long-range dependencies. Subsequent variants such as LSTMs and GRUs alleviated some of these issues but remained constrained by inherently sequential computation [27].

Modern SSM-based architectures aim to retain the favorable inductive bias of recurrence while overcoming the computational limitations of traditional RNNs [6, 11]. Mamba is a prominent example of this new generation of SSMs [11]. Its architecture, illustrated in Figure 2.4, builds upon the classical SSM formulation by introducing a *selective state space mechanism*, in which the parameters governing state updates are dynamically conditioned on the input sequence.

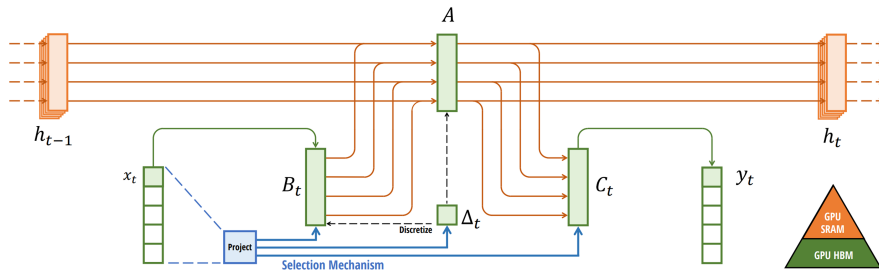


Figure 2.4: Overview of the Mamba selective state space model. Input-dependent projections control the discretization and state update dynamics, enabling selective information flow and hardware-efficient computation.

Unlike the fixed transition dynamics shown in Figure 2.3, Mamba allows the ef-

fective state transition and input matrices to vary as a function of the input. This selectivity enables the model to emphasize relevant parts of the sequence while suppressing irrelevant information, addressing a key limitation of earlier linear SSMs [11]. Crucially, this mechanism preserves linear computational complexity with respect to sequence length, making Mamba well suited for long-context modeling [11, 12].

A key innovation in Mamba is its hardware-aware design, which reformulates state updates to be efficiently computed on modern accelerators [11]. By leveraging structured parameterizations, fused operations, and parallelizable scan algorithms, Mamba achieves high throughput while minimizing memory movement between GPU memory hierarchies. As highlighted in Figure 2.4, this design contrasts sharply with attention-based models, whose quadratic memory access patterns become a bottleneck for long sequences [6].

Mamba-2 extends the original architecture by introducing further optimizations and refinements to the selective state space formulation [14, 27]. These enhancements improve numerical stability, expressivity, and training efficiency, addressing limitations observed in earlier SSM designs. Mamba-2 also places greater emphasis on architectural modularity, enabling smoother integration with other neural components and facilitating hybrid architectures that combine state space dynamics with attention-based modules [14, 42].

Overall, Mamba represents a shift toward sequence models with stronger inductive biases for temporal structure and superior scaling properties for long-context processing [11, 27]. In contrast to Transformers, which excel at global contextual reasoning through attention, Mamba emphasizes efficient, structured memory and sequential dynamics [6]. These complementary strengths motivate the investigation of hybrid architectures that combine attention and state space models, forming the basis for the evolutionary approaches explored in this work.

2.3 Hybrid Sequence Modeling Architectures

Hybrid sequence modeling architectures aim to combine the complementary strengths of attention-based Transformers and State Space Models (SSMs). While Transformers excel at capturing global contextual relationships through self-attention, they suffer from quadratic computational complexity with respect to sequence length [34]. In contrast, SSM-based models such as Mamba offer linear-time sequence processing and strong inductive biases for temporal structure, but may be less effective at modeling complex global interactions [6, 11]. Hybrid architectures seek to reconcile these trade-offs by integrating attention mechanisms and state space dynamics within a unified model.

Recent work has demonstrated that such integration can yield models that are both computationally efficient and highly expressive. Jamba is a prominent example of this design philosophy, combining Transformer attention blocks with Mamba-style selective state space layers within a single architecture [22]. Rather than relying on a uniform stack of identical layers, Jamba interleaves different

types of processing blocks to balance global reasoning and efficient long-range memory.

Figure 2.5 illustrates the high-level block structure employed by Jamba. The architecture consists of a repeated chain of heterogeneous layers, including standard Transformer layers, Mamba layers, and Mixture-of-Experts (MoE) variants of both. Attention-based layers are used sparingly to provide global context aggregation, while Mamba layers dominate the depth of the network, enabling efficient linear-time sequence processing. The inclusion of MoE layers further increases model capacity without proportionally increasing computational cost.

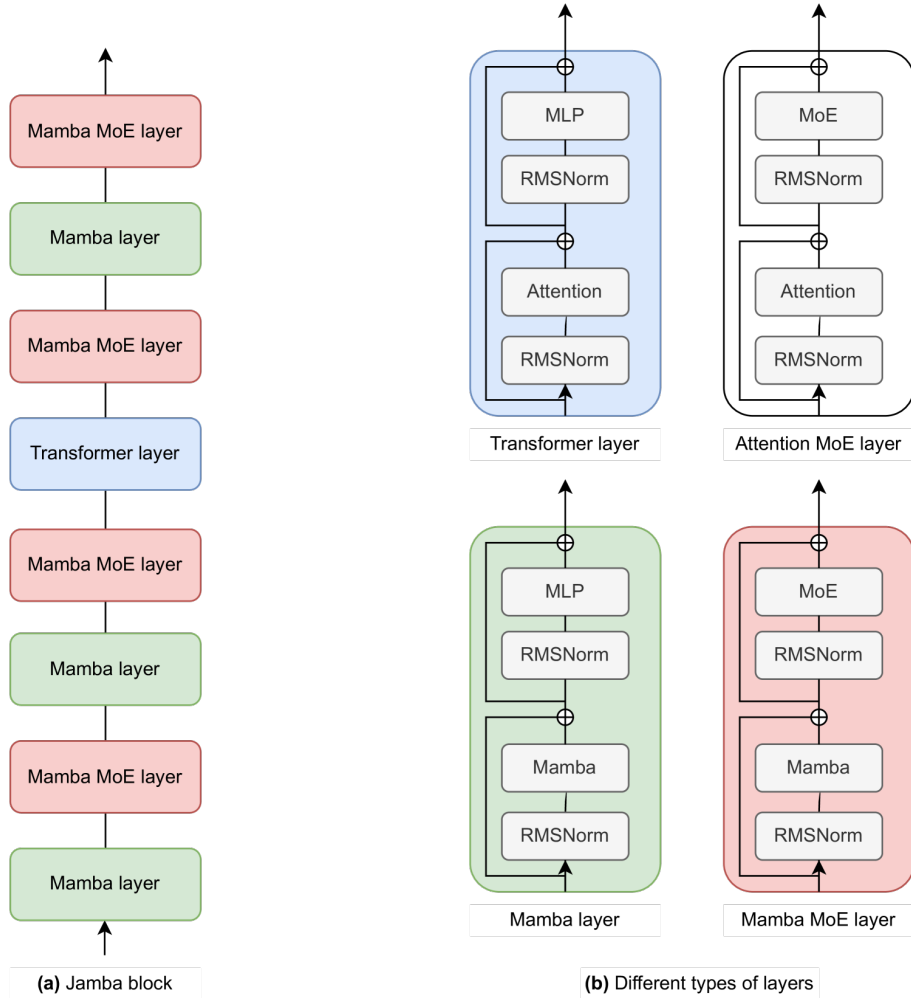


Figure 2.5: High-level block composition of the Jamba architecture. The model interleaves Transformer layers, Mamba layers, and their Mixture-of-Experts (MoE) variants, forming a heterogeneous chain of processing blocks.

This design highlights a key characteristic of modern hybrid architectures: architectural heterogeneity is introduced primarily through the *ordering and frequency* of different block types, rather than through changes to their internal structure. Decisions such as where to place attention layers, how many SSM layers to stack between them, and when to introduce MoE capacity are fixed manually based on expert intuition and empirical validation.

Similarly, Zamba proposes a compact hybrid architecture that blends SSM and

attention modules to achieve competitive performance at reduced parameter and compute budgets [10]. These models illustrate that hybrid designs can outperform purely attention-based or purely SSM-based architectures under practical resource constraints, particularly for long-context tasks. Importantly, both Jamba and Zamba rely on manually designed architectural patterns, reflecting expert assumptions about how different sequence modeling primitives should be composed.

Beyond language modeling, hybridization has also been explored in multi-modal settings. Models such as ML-Mamba and Cobra extend the Mamba framework by incorporating attention-based components to better handle cross-modal interactions, further highlighting the flexibility of hybrid designs [14, 42]. These approaches reinforce the view that attention and state space mechanisms are not competing paradigms, but rather complementary building blocks that can be composed to address diverse modeling requirements.

Despite their promise, existing hybrid architectures remain largely handcrafted, with limited systematic exploration of the design space. Choices such as the ordering of attention and SSM layers, the relative depth allocated to each component, and the granularity of hybridization are typically fixed *a priori*. This results in a vast combinatorial search space that is difficult to explore manually and may contain architectures with superior performance–efficiency trade-offs.

This observation motivates the application of automated architecture search methods to hybrid sequence models. In particular, ENAS offers a natural framework for exploring heterogeneous search spaces composed of multiple architectural primitives. By treating attention and state space modules as evolvable building blocks, evolutionary approaches can systematically discover novel hybrid architectures that balance expressivity, efficiency, and scalability. This perspective underpins the methodology proposed in this work.

2.4 Large Language Models

Large Language Models (LLMs) are deep neural networks trained on massive corpora of text to perform a broad variety of natural language tasks. These models are typically built on Transformer architectures or closely related variants and are trained using large-scale self-supervised objectives, enabling them to capture rich contextual representations and exhibit emergent capabilities beyond traditional task-specific models.

One of the earliest highly influential LLMs was BERT, a bidirectional Transformer-based model that significantly advanced state-of-the-art performance across multiple natural language understanding benchmarks by leveraging deep, contextualized representations learned through masked language modeling and next-sentence prediction [7]. BERT’s success demonstrated the effectiveness of large-scale pre-training on unlabeled text followed by task-specific fine-tuning, establishing a dominant paradigm in modern NLP.

Beyond masked language models, GPT-3 introduced a new scaling-driven approach to autoregressive language modeling, showing that increasing model size leads to strong few-shot and zero-shot performance without explicit fine-tuning [3]. With 175 billion parameters, GPT-3 demonstrated that general-purpose language capabilities can emerge from large-scale pre-training alone, reshaping research priorities toward scale and architectural efficiency.

More recently, the LLM landscape has expanded with models such as LLaMA [33] and BLOOM [18], which explore trade-offs between performance, training efficiency, multilingual coverage, and open-access design. These models have reinforced the role of LLMs as foundational components in modern AI systems while simultaneously highlighting the growing computational, environmental, and engineering costs associated with large-scale model training and deployment.

Despite their impact, the architectures underlying most LLMs remain manually designed, relying on incremental refinements of existing Transformer-based templates. As model scale continues to increase, this manual design process becomes increasingly expensive and restrictive. These challenges motivate the exploration of automated and principled approaches to architectural design, particularly methods capable of discovering efficient and expressive model structures for large-scale language modeling.

2.5 Evolutionary Neural Architecture Search

Neural Architecture Search (NAS) seeks to automate the design of neural network architectures, reducing reliance on manual, expert-driven model engineering. Within this broader field, ENAS leverages principles from EC to explore complex, high-dimensional architecture spaces in a population-based and stochastic manner. Evolutionary approaches are particularly attractive for NAS due to their flexibility, gradient-free optimization, and ability to handle discrete, heterogeneous, and multi-objective design spaces [5].

Unlike gradient-based NAS methods, which often require differentiable search spaces and strong structural constraints, evolutionary methods can naturally operate over diverse architectural components, variable-length representations, and non-differentiable design choices. This makes ENAS well suited for exploring heterogeneous architectures, such as those combining attention-based and state-space-based modules. As a result, evolutionary NAS has been successfully applied across a wide range of domains, including convolutional networks, vision transformers, and large-scale deep learning systems [24, 31].

This section introduces the fundamental concepts of evolutionary algorithms and discusses how they are adapted for neural architecture search, laying the foundation for their application to hybrid sequence modeling architectures in this work.

2.5.1 Evolutionary Algorithms

Evolutionary algorithms (EAs) are a class of population-based optimization methods inspired by natural evolution. They operate by maintaining a population of candidate solutions, which evolve over successive generations through the application of variation operators such as mutation and crossover, guided by a fitness-based selection process [13, 32]. Over time, this iterative process biases the population toward increasingly fit solutions.

Figure 2.6 illustrates the canonical evolutionary algorithm cycle. The process begins with the initialization of a population of candidate solutions, which are evaluated according to a fitness function. Based on these evaluations, a selection mechanism chooses parent individuals that are then modified by variation operators to produce offspring. A replacement strategy determines how offspring are incorporated into the population, forming a new generation. This cycle is repeated until a termination criterion is met, with the fittest individuals emerging over successive generations.

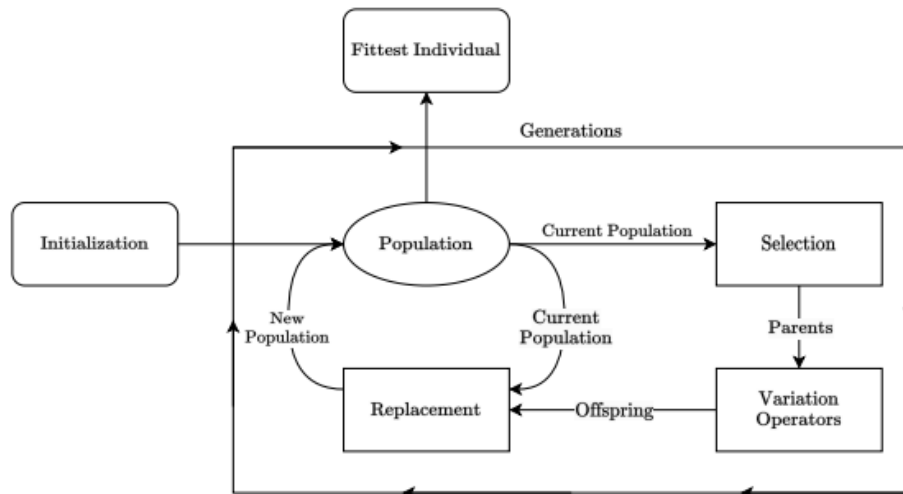


Figure 2.6: Evolutionary Algorithm Cycle.

A defining characteristic of evolutionary algorithms is their reliance on fitness evaluations rather than gradient information. This property allows EAs to optimize objectives that are non-differentiable, noisy, multi-modal, or computationally expensive. As a result, evolutionary methods have been widely adopted in settings where traditional optimization techniques struggle, including reinforcement learning, control, and neural architecture optimization [29, 37].

Several families of evolutionary algorithms are particularly relevant in the context of neural architecture search. Genetic Algorithms (GAs) typically represent candidate solutions as discrete genomes and apply crossover and mutation operators to explore the search space. Evolution Strategies (ES), in contrast, focus on optimizing real-valued parameters through stochastic perturbations and population-level statistics, offering strong scalability properties [29]. Neuroevolution approaches, such as NEAT, explicitly evolve both the topology and parameters of neural networks, enabling the discovery of novel architectures without

predefined structural constraints [32].

Beyond single-objective optimization, evolutionary algorithms naturally extend to multi-objective settings, where trade-offs between competing goals—such as accuracy, computational cost, memory footprint, and latency—must be considered simultaneously. Population-based methods can approximate Pareto fronts of optimal solutions, making them especially well suited for architecture search under real-world constraints [15, 24]. These properties form the methodological basis for ENAS approaches discussed in the following subsection.

2.5.2 Evolutionary Algorithms Applied to Neural Architecture Search

ENAS applies evolutionary algorithms to the automated discovery of neural network structures, encoding architectural decisions into evolutionary representations and optimizing them through population-based search. Compared to gradient-based NAS methods, evolutionary approaches offer greater flexibility in defining search spaces, supporting discrete, variable-length, and heterogeneous architectural representations without requiring differentiability [5].

Genetic Algorithms for NAS. Genetic Algorithms (GAs) are among the earliest and most widely used evolutionary techniques for architecture search. In GA-based NAS, candidate architectures are typically encoded as genomes describing layer types, connectivity patterns, or hyperparameters, which are evolved through mutation and crossover operators [32]. Early work demonstrated that evolutionary methods could discover competitive neural architectures without manual design, a paradigm later scaled to deep convolutional networks and large search spaces [28].

In modern NAS settings, GAs have been applied to model compression, pruning, and efficiency-aware optimization. Methods such as EvoPress and EAPruning evolve architectures under resource constraints, explicitly balancing predictive performance against model size, latency, or energy consumption [21, 30]. These approaches highlight the suitability of genetic search for multi-objective architecture optimization.

Evolution Strategies and Scalable NAS. Evolution Strategies (ES) represent an alternative evolutionary paradigm that emphasizes scalability and parallelism. Rather than relying on discrete crossover operations, ES methods optimize architectures through stochastic perturbations and population-level statistics [29]. ES-based NAS has been shown to scale effectively to large models and distributed computing environments, making it attractive for architecture search in high-dimensional spaces [37].

Population Based Training (PBT) further extends this idea by jointly optimizing model parameters and hyperparameters during training, periodically exchanging information between population members [15]. Multi-objective variants of

PBT enable the exploration of trade-offs between accuracy and efficiency, which is particularly relevant for large-scale neural architectures [19].

Multi-population and Structured Evolutionary Search. To improve exploration and avoid premature convergence, several ENAS methods employ multiple interacting populations or structured evolutionary dynamics. Multi-population NAS frameworks alternate between exploration-focused and exploitation-focused populations, enabling more efficient coverage of large and rugged search spaces [41]. Similarly, approaches such as EG-NAS and EG-ENAS introduce fast evolutionary exploration strategies that decouple architectural mutation from expensive full training [38, 43].

More recently, hybrid evolutionary frameworks have been proposed that integrate evolutionary search with generative or training-free evaluation mechanisms. For example, Evolution Meets Diffusion explores neural architecture generation through diffusion-based priors guided by evolutionary objectives, further expanding the scope of ENAS methodologies [23].

Applications to Sequence Models and Transformers. Evolutionary NAS has increasingly been applied to sequence models, including Transformers and large language models. The Evolved Transformer demonstrated that evolutionary search can discover competitive attention-based architectures without manual design [31]. Subsequent work has explored evolutionary pruning, architectural adaptation, and efficiency optimization for Transformer-based models in both vision and language domains [9, 24].

Despite these advances, most existing ENAS approaches for sequence modeling remain constrained to homogeneous Transformer architectures. Only limited work has explored evolutionary optimization in heterogeneous or hybrid architecture spaces, particularly those combining attention mechanisms with alternative sequence modeling paradigms. This limitation motivates the exploration of evolutionary methods for hybrid Transformer-Mamba architectures pursued in this dissertation.

2.6 State of the Art

This section reviews prior work at the intersection of EC and neural architecture design for sequence models. It focuses on ENAS for Transformer architectures, evolutionary optimization techniques applied to large language models, and emerging efforts that move beyond Transformer-only design spaces. By situating recent advances within these themes, this section identifies key limitations of existing approaches and motivates the need for evolutionary methods capable of exploring hybrid and heterogeneous sequence modeling architectures.

2.6.1 Evolutionary NAS for Transformer Architectures

ENAS has been extensively explored in the context of Transformer-based models, primarily with the goal of improving performance, efficiency, and architectural adaptability while reducing reliance on manual design. Early work demonstrated that evolutionary methods are capable of discovering competitive attention-based architectures without relying on fixed Transformer templates.

A seminal contribution in this area is the *Evolved Transformer*, which applied evolutionary search to the design of encoder–decoder attention architectures for sequence-to-sequence tasks [31]. By evolving the structure of attention blocks, feedforward components, and connectivity patterns, the approach identified architectures that matched or exceeded the performance of hand-designed Transformers while using fewer parameters. This work provided early evidence that evolutionary algorithms can effectively navigate the complex architectural design space of attention-based models.

Subsequent research expanded evolutionary NAS to explore a broader range of Transformer design dimensions, including depth, width, attention head configurations, and block-level composition. Several approaches focused on multi-objective optimization, balancing predictive performance against computational cost, memory footprint, or latency constraints [24, 44]. These methods demonstrated that evolutionary search can identify Pareto-optimal Transformer variants that achieve favorable trade-offs between accuracy and efficiency.

Evolutionary techniques have also been applied to structural pruning and compression of Transformer architectures. Methods such as evolutionary pruning and dynamic compression evolve layer-level or block-level removal strategies, enabling substantial reductions in model size while maintaining performance [21, 30]. Related work has explored evolutionary adaptation of Transformer architectures under resource constraints, particularly in vision and multimodal settings [20].

Beyond static architecture design, evolutionary approaches have been combined with population-based training and evolution strategies to optimize Transformer training dynamics and hyper parameters [15, 25]. These methods emphasize adaptability and robustness during training rather than fixed architectural discovery, highlighting the versatility of EC in Transformer optimization.

Comprehensive surveys of Transformer NAS methods indicate that most existing evolutionary approaches operate within largely homogeneous Transformer search spaces, focusing on variations of attention-based blocks and feedforward components [36, 40]. While effective, this focus inherently limits the exploration of fundamentally different sequence modeling mechanisms.

Overall, the state-of-the-art demonstrates that evolutionary NAS is a powerful tool for optimizing Transformer architectures, enabling automated discovery of efficient and high-performing attention-based models. However, the vast majority of existing work remains confined to pure Transformer design spaces, leaving heterogeneous and hybrid architectures—particularly those combining attention with alternative sequence modeling paradigms—largely unexplored. This

gap motivates the investigation of evolutionary search methods beyond standard Transformer architectures, as pursued in this dissertation.

2.6.2 Evolutionary Optimization of Large Language Models

As large language models scale to billions of parameters, direct evolutionary search over full architectures becomes computationally prohibitive. Consequently, recent research has shifted toward applying EC techniques to *optimize, adapt, and combine* pre-trained LLMs rather than evolving them from scratch. In this setting, EC is primarily employed at higher levels of abstraction, such as model composition, weight adaptation, hyperparameter evolution, and training dynamics.

One prominent line of work explores evolutionary optimization for *model merging and weight-space manipulation*. Akiba et al. introduced evolutionary optimization of model merging recipes, where populations of candidate merge configurations are evolved to combine multiple pre-trained models into a single performant model without additional large-scale training [1]. This approach demonstrates that evolutionary search can effectively navigate complex weight-space interactions, yielding merged LLMs that outperform manually designed merging strategies.

Closely related to this idea, Du et al. proposed evolving weight combinations for knowledge fusion across language models [8]. Rather than designing fusion mechanisms heuristically, evolutionary algorithms are used to search for optimal weightings that integrate complementary knowledge from multiple models. These methods highlight the suitability of EC for optimization problems where gradients are unavailable or unreliable, and where evaluation can be performed at the level of downstream task performance.

Beyond weight-level optimization, evolutionary methods have been investigated for *structural adaptation and pruning* of large language models. Structural NAS techniques have been applied to identify redundant layers or components within LLMs, enabling parameter reduction while preserving performance [16]. Although constrained to relatively coarse-grained architectural modifications, these approaches demonstrate that evolutionary search remains viable even in large-scale model regimes when the search space is carefully restricted.

More broadly, recent surveys emphasize that EC offers complementary advantages to gradient-based optimization when applied to LLMs, particularly in settings involving discrete design choices, multi-objective trade-offs, or black-box evaluation pipelines [4, 35]. Evolutionary strategies have been proposed for optimizing prompts, training curricula, hyperparameters, and even interaction policies between LLMs and external systems, extending the applicability of EC well beyond architecture search alone.

Despite these advances, most evolutionary optimization techniques for LLMs operate *around* fixed architectures rather than within the architectural design space itself. The dominant focus remains on Transformers, and only limited work explores evolutionary optimization in alternative or hybrid sequence modeling

paradigms. As a result, the application of evolutionary methods to heterogeneous architectures—particularly those integrating attention-based models with state space dynamics—remains an open and underexplored research direction.

In summary, EC has emerged as a practical and flexible tool for optimizing large language models at scale, enabling effective exploration of weight-space, structural, and training-level design choices. However, its potential for guiding architectural innovation beyond standard Transformer designs has yet to be fully realized, motivating the investigation of evolutionary optimization in hybrid and non-attention-based LLM architectures pursued in this work.

2.6.3 Evolutionary Search Beyond Transformers

While the Transformer architecture has dominated sequence modeling research for several years, its limitations in terms of computational efficiency, memory usage, and scalability have motivated the exploration of alternative sequence modeling paradigms. In response, recent work has begun to investigate architectures that move beyond pure self-attention, including state space models (SSMs), recurrent hybrids, convolutional sequence models, and other attention-free or partially attention-based designs. However, ENAS in these broader architectural spaces remains comparatively underexplored.

Early evolutionary NAS frameworks were largely developed for convolutional and recurrent neural networks, focusing on homogeneous architecture families with well-understood inductive biases. As a result, most evolutionary methods for sequence modeling have inherited a strong architectural bias toward Transformer-style designs, often restricting the search space to variations in attention heads, feed-forward widths, or layer depth. While effective for incremental improvements, such constraints limit the discovery of fundamentally different sequence modeling strategies.

Recent advances in state space models, such as S4 and Mamba, have demonstrated that competitive sequence modeling performance can be achieved without explicit attention mechanisms, offering linear-time complexity and improved efficiency on long sequences. Despite their growing adoption in hand-designed architectures, these models have seen limited integration into evolutionary NAS pipelines. Existing evolutionary approaches rarely consider heterogeneous search spaces that allow the combination of attention-based and state-space-based components within a single architecture.

A small number of works have begun to move in this direction by incorporating non-attention modules into NAS search spaces or by exploring hybrid architectures through manual design. However, these efforts typically rely on gradient-based NAS or heuristic design choices rather than population-based evolutionary search. Consequently, the potential of EC to navigate complex, discrete, and heterogeneous architecture spaces—where gradient signals are weak or unavailable—remains largely untapped.

This gap highlights a key opportunity for evolutionary NAS: enabling the au-

tomated discovery of hybrid sequence modeling architectures that combine complementary modeling paradigms. By extending evolutionary search beyond Transformer only spaces, it becomes possible to explore architectural trade-offs between expressivity, efficiency, and scalability in a principled and systematic manner. Addressing this challenge is particularly relevant for modern large language models, where architectural efficiency and modularity are increasingly important constraints.

2.6.4 Summary of Limitations and Research Gaps

The literature reviewed in the preceding sections demonstrates substantial progress in both EC for neural architecture search and the development of efficient alternatives to Transformer-based sequence modeling. However, when these two research directions are considered jointly, several limitations emerge that motivate further investigation.

First, evolutionary approaches applied to Transformer models typically operate within narrowly constrained architectural search spaces. Most methods focus on optimizing parameters such as depth, width, attention configurations, or pruning strategies while preserving the core self-attention paradigm. While effective for refining existing architectures, this constraint fundamentally limits the ability of evolutionary search to discover qualitatively different sequence modeling mechanisms.

Second, as language models scale to billions of parameters, EC has increasingly been applied at higher levels of abstraction. Recent work emphasizes model merging, weight adaptation, hyperparameter evolution, and training dynamics, deliberately avoiding direct architectural evolution due to its prohibitive computational cost. Consequently, architectural innovation in large language models has largely remained decoupled from evolutionary search.

Third, recent advances in state space models and hybrid Transformer-SSM architectures demonstrate that attention-free or partially attention-based designs can achieve competitive performance with improved efficiency. Models such as Mamba and hybrid architectures like Jamba highlight the viability of alternative sequence modeling paradigms. However, these architectures are almost exclusively the result of manual design and expert-driven iteration. To date, evolutionary methods have seen limited application in exploring heterogeneous architecture spaces that combine attention-based and state-space-based components.

Taken together, these observations expose a broader research gap: the absence of evolutionary methods capable of systematically exploring hybrid sequence modeling architectures that extend beyond Transformer-only designs while remaining computationally feasible. Addressing this gap in full would require evolutionary optimization over large-scale heterogeneous architecture spaces combined with extensive pretraining or fine-tuning at LLM scale, an undertaking that realistically spans years of research and substantial computational resources.

In this work, we explicitly acknowledge the scope of this broader challenge while

deliberately focusing on a constrained and tractable subset of the problem. Rather than evolving large language models from scratch or conducting unconstrained architecture search, we investigate evolutionary strategies for exploring restricted hybrid architecture spaces that combine Transformer and Mamba components under practical computational budgets. This design choice enables a focused study of the interaction between evolutionary search and hybrid sequence modeling while remaining feasible within the timeframe of a master’s thesis.

Table 2.1 summarizes the key characteristics of the representative sequence modeling approaches discussed in this chapter. The comparison highlights a clear gap in the existing literature: while evolutionary methods have been successfully applied to Transformer architectures, no prior work jointly integrates EC with hybrid Transformer-Mamba sequence modeling architectures.

Table 2.1: Comparison of representative sequence modeling approaches.

Approach	Uses EC	Transformer	Mamba	Hybrid
Evolved Transformer [31]	✓	✓	–	–
Mamba [11]	–	–	✓	–
Jamba [22]	–	✓	✓	✓
This work	✓	✓	✓	✓

As illustrated in Table 2.1, existing approaches typically satisfy only a subset of these criteria. Evolutionary methods remain largely confined to Transformer-centric design spaces, while hybrid architectures rely on manual construction. The approach pursued in this thesis aims to bridge this gap by applying evolutionary search to a constrained hybrid architecture space, balancing architectural expressivity with computational feasibility.

Chapter 3

Proposed Methodology

This work adopts an exploratory and experimental methodology aimed at assessing the feasibility and effectiveness of EC for neural architecture search in hybrid large language model architectures. In particular, the methodology targets several open problems in current language model design: (i) the reliance on manually specified architectures when combining heterogeneous sequence modeling paradigms, (ii) the limited understanding of how attention-based and state-space-based components should be composed for different downstream tasks, and (iii) the difficulty of identifying architectures that balance performance and computational efficiency under realistic constraints. Given the novelty of combining evolutionary methods with heterogeneous sequence modeling paradigms, and the limited development time available, the proposed approach prioritizes conceptual clarity, experimental control, and incremental validation over exhaustive architectural complexity. A high-level overview of the proposed methodology is presented in Figure 3.1.

At the core of the methodology (Figure 3.1) is a modular representation of language model architectures as ordered sequences of processing blocks. Each block corresponds to a specific sequence modeling mechanism, namely attention-based blocks derived from the Transformer architecture and state-space-based blocks inspired by the Mamba model. An individual architecture is therefore defined as a chain of blocks, where each position specifies the type of block instantiated at that layer. This representation naturally supports pure Transformer models, pure Mamba models, and a wide range of hybrid configurations that interleave both paradigms within a single network.

Based on this representation, an evolutionary algorithm is employed to explore the resulting architectural design space, as illustrated in Figure 3.1. Each individual in the population corresponds to a distinct block sequence, and evolutionary variation is applied directly at the architectural level. Selection is driven by task performance, while variation is introduced through standard evolutionary operators acting directly on the block sequence representation. Crossover is implemented as an operator that exchanges contiguous subsequences of blocks between parent architectures, while mutation is an operation that changes the block type at a given layer position (i.e., replacing an attention block with a state-space

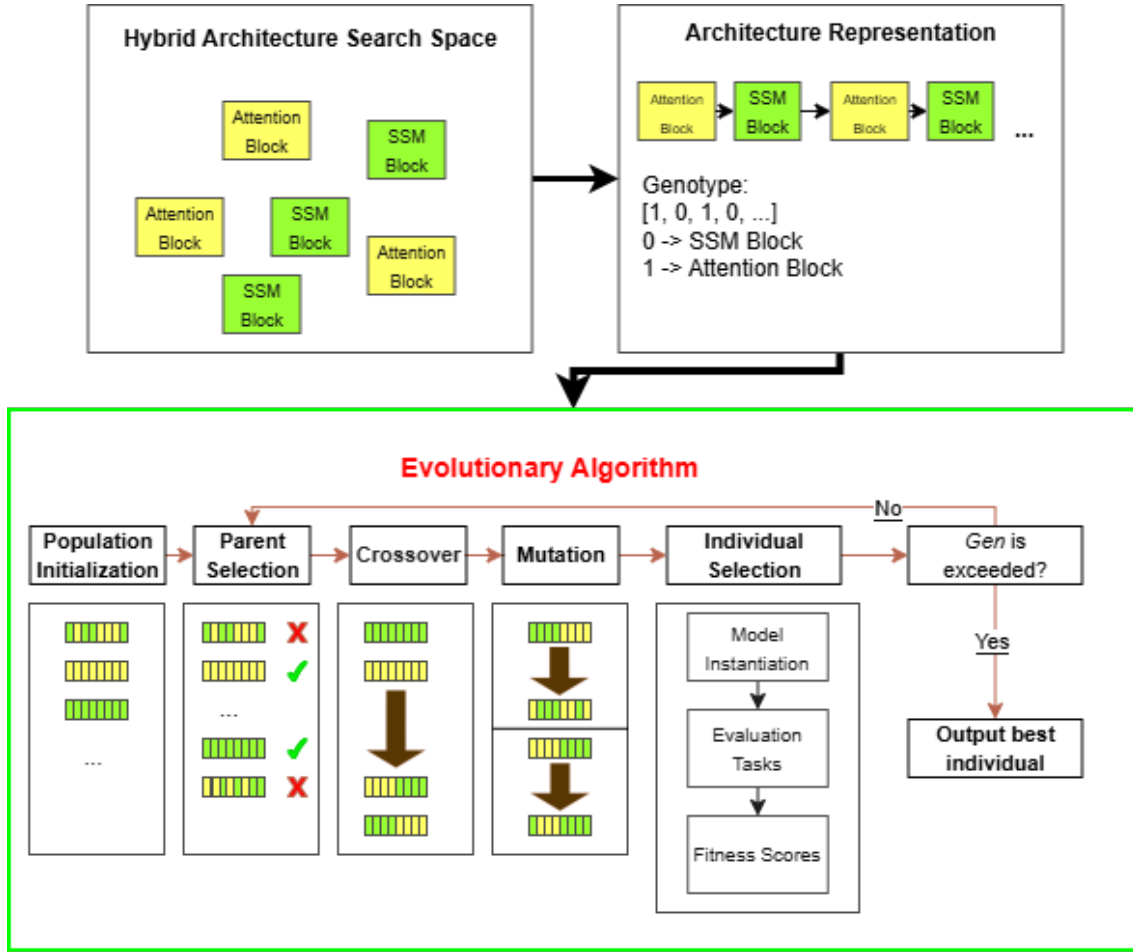


Figure 3.1: High-level overview of the proposed methodology.

block or vice versa). This setup enables the evolutionary process to search beyond manually designed hybrids and to discover potentially non-obvious combinations of attention-based and state-space-based components.

Initially, the architectures will be defined with a fixed total number of layers L , where each layer is encoded as a discrete gene taking one of two possible values: an attention-based block or a state-space-based block. Under this formulation, the architectural search space consists of 2^L possible configurations, enabling controlled exploration while keeping the combinatorial complexity tractable.

Given the high computational cost associated with training large language models, the evolutionary search is conducted under constrained evaluation regimes. Various techniques for reducing training cost and stabilizing fitness estimation—such as partial training, parameter reuse, and progressive evaluation—will be explored where appropriate, with the goal of maintaining a practical balance between computational efficiency and informative fitness signals.

The evolved architectures are evaluated on downstream language tasks that are relevant to the objectives of the AMALIA project. These include document classification and text summarization, as well as machine translation into European Portuguese. For document classification, benchmark datasets such as AG News, which was already used in the preliminary work, as well as other widely adopted

datasets for text classification, are considered as representative evaluation tasks, with experiments conducted on Portuguese-language datasets or Portuguese subsets where available, in line with the AMALIA training setting.

For text summarization, standard benchmarks commonly used in neural summarization research, such as CNN/DailyMail or XSum, are used as conceptual and methodological references, while Portuguese summarization datasets, including news and document-level corpora available in the AMALIA context or publicly available Portuguese resources, are prioritized for empirical evaluation. This ensures comparability with existing work while maintaining linguistic consistency with the project’s objectives.

For machine translation, the focus is placed on translation into European Portuguese, leveraging datasets and benchmarks commonly used in multilingual and variety-aware translation settings. These include parallel corpora from widely used resources such as Europarl, OPUS-based datasets, and multilingual benchmarks such as FLORES, which include European Portuguese and enable consistent comparison across languages. Evaluation protocols follow established practice in machine translation, with task-specific metrics selected according to the nature of each task.

Each task is evaluated using its standard performance metrics, such as accuracy and F1-score for document classification, ROUGE for text summarization, and BLEU, ROUGE, or learned metrics such as COMET for machine translation. This task-specific evaluation ensures that evolved architectures are assessed using metrics that reflect the different objectives and quality criteria of each application scenario.

Performance of evolved architectures is compared against manually specified baselines, including Transformer-only, Mamba-only, and fixed hybrid configurations. These comparisons aim to isolate the contribution of evolutionary search from architectural choices defined a priori. All models, training procedures, and evolutionary operators are implemented using the PyTorch framework, enabling modular experimentation and facilitating reproducibility of the proposed approach.

The experimental analysis focuses on understanding whether EC can effectively operate in heterogeneous architectural spaces and whether it can identify competitive hybrid architectures under realistic constraints. Rather than aiming for state-of-the-art results, the goal is to assess the viability, stability, and exploratory power of evolutionary search in this setting. The findings are expected to provide empirical evidence regarding the applicability of evolutionary methods to hybrid sequence models and to highlight directions for future research involving richer representations, more expressive variation operators, and large-scale training scenarios.

Given the exploratory nature of this work and the limited development time available, the methodology described above and summarized in Figure 3.1 is considered the primary research plan. It is intentionally designed to be conceptually simple and computationally tractable, while still addressing the core research questions of this dissertation.

If the proposed approach proves viable and sufficient resources are available, additional extensions can be explored. These include richer architectural representations that allow variation beyond block type, multi-level or grammar-based encodings inspired by prior work in evolutionary architecture search, and more expressive variation operators that act at different abstraction levels. Such extensions would aim to increase the diversity of the search space and to further assess the scalability of evolutionary methods in heterogeneous model design.

Nevertheless, the fundamental objective of this dissertation remains the evaluation of EC as a practical and effective tool for exploring hybrid Transformer–Mamba architectures, and all proposed extensions are considered complementary rather than essential to accomplishing this goal.

Chapter 4

Preliminary Work

This chapter presents the work developed during the first semester of this dissertation. The main objective of this phase was to acquire a solid conceptual and practical understanding of modern language modeling architectures, as well as to explore the feasibility of applying EC to hybrid sequence modeling designs. Consequently, the work carried out in this period is exploratory in nature and serves as a foundation for the subsequent stages of the dissertation.

The first stage of the work focused on understanding the internal mechanisms of language models and their underlying architectural components. To this end, simplified implementations of two representative architectures were developed from scratch: a decoder-only Transformer model and a state space model inspired by Mamba.

These implementations were not intended for direct comparison or benchmarking. Different datasets and training setups were used, and no systematic evaluation was performed. Instead, the objective was to gain hands-on experience with attention-based and state-space-based sequence modeling, including their implementation details, training behavior, and architectural trade-offs. This practical work complemented the theoretical background study and informed later architectural design choices.

After gaining familiarity with pure Transformer and pure state space architectures, the focus shifted to hybrid sequence modeling designs that combine both paradigms. For this purpose, a simplified implementation inspired by the Jamba architecture was adopted as a structural baseline.

The hybrid model was designed as a stack of modular layers, where each layer can operate either as an attention-based block or as a state space block. This modular abstraction enables flexible composition of heterogeneous architectures and serves as the basis for architectural manipulation through EC.

Building on the modular hybrid architecture, a prototype evolutionary loop was implemented to explore different layer-level configurations of attention and state space components. Each candidate architecture is represented by a discrete genotype, where each gene encodes the type of computation used at a given layer.

Figure 3.1 provides a high-level overview of the proposed methodology, illustrating the hybrid architecture search space, the genotype-to-architecture mapping, and the evolutionary process used to explore this space.

The evolutionary procedure follows a standard genetic algorithm structure. An initial population of architectures is created, including homogeneous and heterogeneous configurations. Each individual is instantiated by activating the corresponding layer types, trained for a small number of steps on a downstream task, and evaluated using validation accuracy as a fitness measure. Selection, crossover, and mutation are then applied at the layer level to generate new candidate architectures.

The evolutionary process is intentionally constrained: training budgets are limited, most model parameters are frozen, and the search space is restricted to layer-type selection. The goal at this stage is not to discover optimal architectures, but to validate the feasibility of evolutionary exploration over hybrid sequence modeling designs.

The prototype evolutionary experiments were conducted on a news topic classification task, where input texts are assigned to one of four categories. A reduced version of the AG News dataset was used, with 200 training samples and 200 validation samples, allowing rapid evaluation of multiple architectures. A resume of the experimental setup used is represented in Table 4.1.

This task was selected because it resembles downstream classification scenarios relevant to the AMALIA project, while remaining computationally lightweight.

Table 4.1: Prototype experimental setup.

Component	Configuration
Base architecture	Pretrained Jamba
Architecture encoding	Layer-type genotype
Population size	40
Generations	10
Selection	Tournament (size 6)
Elitism	5 %
Crossover Rate	1.0
Mutation Rate	0.1
Dataset	AG News (subset)
Training samples	200
Validation samples	200
Training regime	Partial fine-tuning (classifier and norms)
Fitness	Validation Accuracy

Despite the simplified setup, the evolutionary process consistently produced diverse hybrid architectures with measurable differences in validation accuracy across generations. Mixed attention-state space configurations often outperformed homogeneous designs under identical training constraints, indicating that the architectural composition has a non-trivial impact on downstream performance.

These results are not intended as conclusive empirical evidence, but rather as a proof of concept demonstrating that EC can be applied to hybrid Transformer-Mamba architectures in a practical and computationally feasible manner.

The work presented in this chapter represents only the first semester of the dissertation and is deliberately limited in scope. The evolutionary search operates over a restricted architecture space, training is minimal, and the downstream task is simplified. Nevertheless, this exploratory work establishes a concrete technical foundation for the subsequent stages of the dissertation, motivating more systematic investigation of evolutionary hybrid architecture search under realistic constraints.

Chapter 5

Work Plan and Timeline

This chapter outlines the planned work for the second semester of the dissertation, covering the period from **9 February to 6 July**. Building on the exploratory work conducted during the first semester, the focus of this phase is on implementing, evaluating, and documenting the proposed evolutionary hybrid architecture search methodology, culminating in the submission and defense of the final dissertation.

The work plan is structured around clearly defined milestones, with parallel activities where appropriate. In addition to technical development, sufficient time is allocated for report writing, revision, and preparation for the final presentation.

The remaining work is organized into the following main phases:

- **Methodology Refinement and Design** Refinement of the evolutionary framework, definition of the hybrid architecture search space, fitness objectives, and evaluation protocol.
- **Implementation and Experimental Setup** Extension of the prototype evolutionary framework into a more systematic experimental pipeline, including dataset preparation, training protocols, and logging infrastructure.
- **Experimental Evaluation** Execution of controlled experiments to evaluate the behavior of evolutionary search over hybrid architectures, analysis of performance trends, and comparison with baseline configurations.
- **Analysis and Interpretation** Quantitative and qualitative analysis of results, identification of architectural patterns, limitations, and implications for hybrid sequence modeling.
- **Writing and Documentation** Progressive writing of the dissertation.
- **Revision and Presentation Preparation** Final revision of the document, incorporation of feedback, and preparation of the dissertation defense presentation.

Continuous background reading and review of relevant literature will be conducted throughout the semester to support design decisions and contextualize results.

Figure 5.1 presents a Gantt chart summarizing the planned timeline and task dependencies for the second semester.

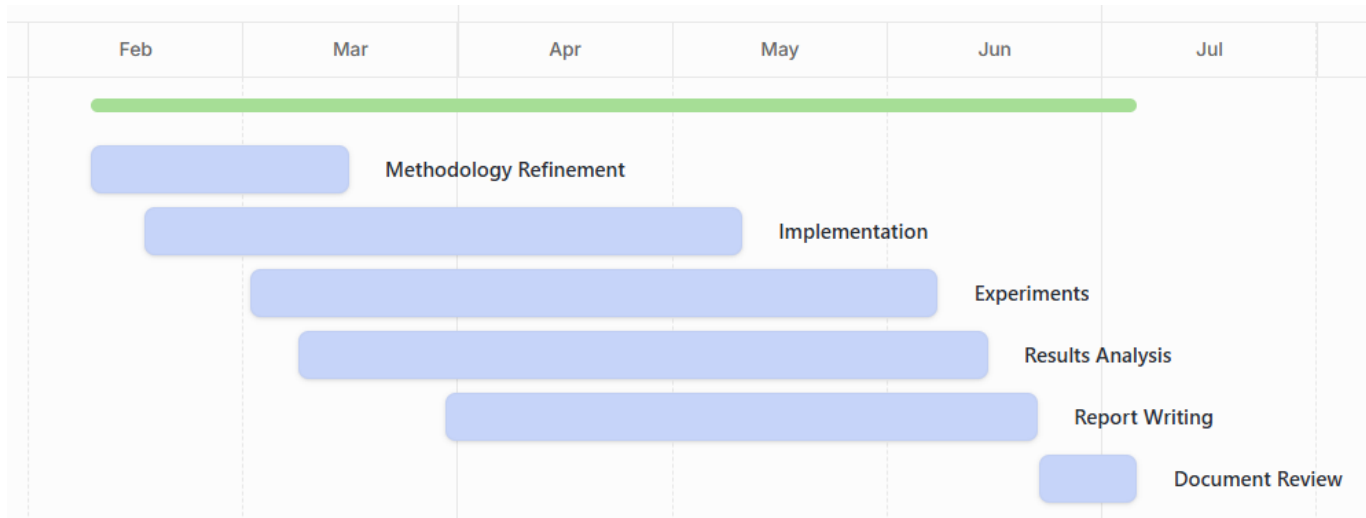


Figure 5.1: Planned work schedule for the second semester of the dissertation.

To ensure traceability and project management discipline, the Gantt chart is complemented by a github task backlog maintained using version control tools. A snapshot of this backlog is provided in Figure 5.2, illustrating how high-level milestones are translated into concrete implementation tasks.

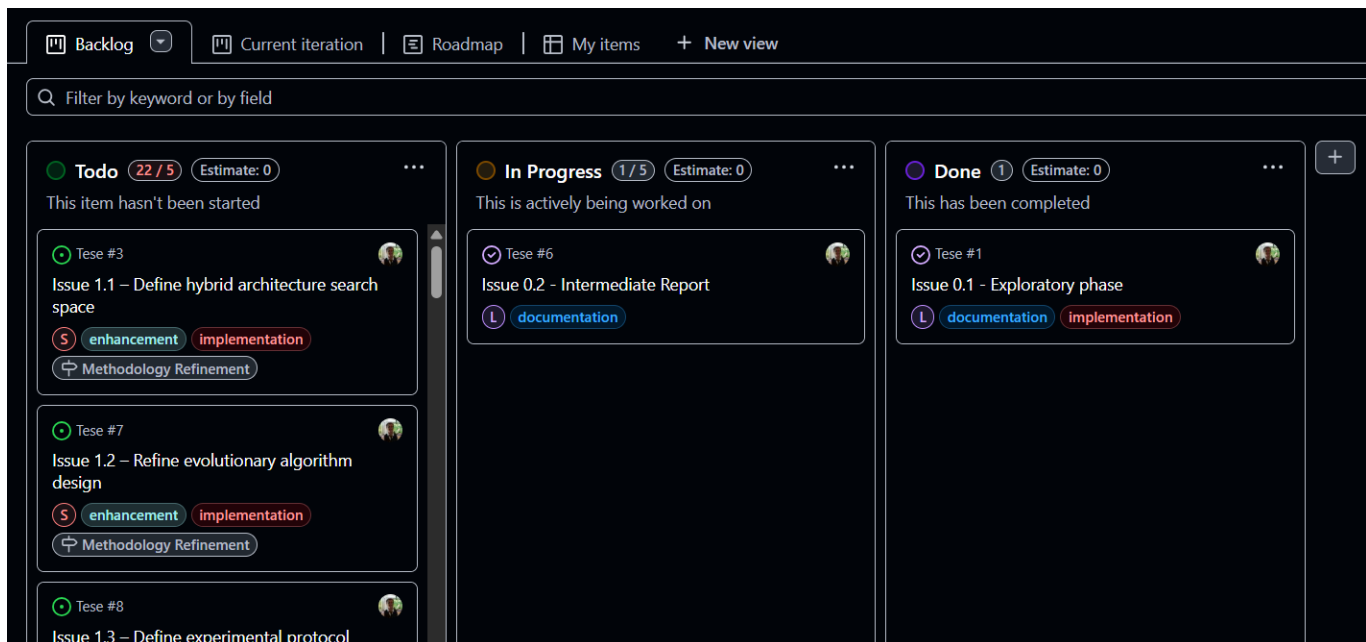


Figure 5.2: GitHub issue backlog used to organize and track the remaining work of the dissertation.

A dedicated buffer of approximately two weeks is reserved at the end of the

schedule to accommodate final revisions and defense preparation.

Several potential risks have been identified for the remaining work, along with corresponding mitigation strategies:

- **Risk: High computational cost of evolutionary experiments** *Mitigation:* Restrict search space complexity, limit training budgets, freeze model parameters where possible, and prioritize small- to medium-scale experiments.
- **Risk: Experimental results lack clear trends** *Mitigation:* Focus on controlled ablation studies, emphasize qualitative insights, and clearly frame results as exploratory where appropriate.
- **Risk: Time constraints due to implementation complexity** *Mitigation:* Maintain a prioritized task backlog, focus on core contributions, and avoid unnecessary extensions beyond the defined scope.
- **Risk: Delays in writing or integration of results** *Mitigation:* Write sections incrementally alongside experimentation and reserve explicit time for revision and consolidation.

Overall, the proposed plan balances ambition with feasibility and includes sufficient flexibility to adapt to unforeseen challenges while ensuring timely completion of the dissertation.

Chapter 6

Conclusion

With the rapid development of large language models, new architectural paradigms have emerged to address limitations of Transformer-based sequence modeling, particularly in terms of efficiency and scalability. At the same time, EC has shown strong potential for exploring neural architectures, yet its application to modern hybrid language model designs remains limited.

The work carried out during this semester contributed to a deeper understanding of both research directions. An extensive study of Transformers, state space models, hybrid architectures, and ENAS was conducted, providing the necessary theoretical background for the proposed research. In parallel, practical implementations of representative architectures were developed to gain hands-on familiarity with their internal mechanisms and design trade-offs.

Building on this foundation, a preliminary evolutionary prototype was implemented to explore hybrid Transformer-Mamba architectures in a constrained and computationally feasible setting. Although exploratory in nature, this work demonstrated the viability of applying evolutionary strategies to hybrid sequence modeling designs and helped identify key challenges and design choices for future experimentation.

In conclusion, this document presents the work developed during the first semester, combining background study, state-of-the-art analysis, and initial practical experimentation. The results obtained establish a clear direction for the second semester, during which the evolutionary framework will be refined, experiments will be expanded, and a more systematic evaluation of hybrid architectures will be conducted.

References

- [1] Takuya Akiba, Makoto Shing, Yujin Tang, Qi Sun, and David Ha. Evolutionary optimization of model merging recipes. *Nature Machine Intelligence*, 7(8):177–188, 2025. URL <https://www.nature.com/articles/s42256-024-00975-8>.
- [2] Jay Alammar. The illustrated transformer, 2018. URL <https://jalammar.github.io/illustrated-transformer/>.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. Language models are few-shot learners, 2020. URL <https://arxiv.org/abs/2005.14165>. arXiv:2005.14165.
- [4] Dikshit Chauhan, Bapi Dutta, Indu Bala, Niki van Stein, Thomas Bäck, and Anupam Yadav. Evolutionary computation and large language models: A survey of methods, synergies, and applications, 2025. URL <https://arxiv.org/abs/2505.15741>. arXiv:2505.15741.
- [5] Krishna Chitty-Venkata, Arjun Somani, et al. A survey of neural architecture search. *ACM Computing Surveys*, 2022.
- [6] Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality, 2024. URL <https://arxiv.org/abs/2405.21060>. arXiv:2405.21060.
- [7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2018. URL <https://arxiv.org/abs/1810.04805>. arXiv:1810.04805.
- [8] Guodong Du, Jing Li, Hanting Liu, Runhua Jiang, Shuyang Yu, Yifei Guo, Sim Kuan Goh, and Ho-Kin Tang. Knowledge fusion by evolving weights of language models, 2024. URL <https://arxiv.org/abs/2406.12208>. arXiv:2406.12208.
- [9] Wenqi Fan, Haohao Qu, and Tyler Derr. Training-free model architecture search for mamba. In *NeurIPS*, 2025. URL <https://openreview.net/forum?id=8q2kReYRDn>.
- [10] Paolo Glorioso, Quentin Anthony, Yury Tokpanov, James Whittington, Jonathan Pilault, Adam Ibrahim, and Beren Millidge. Zamba: A compact 7b ssm hybrid model, 2024. URL <https://arxiv.org/abs/2405.16712>. arXiv:2405.16712.

- [11] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023. URL <https://arxiv.org/abs/2312.00752>. arXiv:2312.00752.
- [12] John Timothy Halloran, Manbir S. Gulati, and Paul F. Roysdon. Mamba state-space models can be strong downstream learners. In *NeurIPS*, 2024. URL <https://openreview.net/forum?id=C3t6GMPnC5>.
- [13] Nikolaus Hansen. *The CMA Evolution Strategy: A Tutorial*. arXiv, 2016. URL <https://arxiv.org/abs/1604.00772>. arXiv:1604.00772.
- [14] Wenjun Huang, Jiakai Pan, Jiahao Tang, Yanyu Ding, Yifei Xing, Yuhe Wang, Zhengzhuo Wang, and Jianguo Hu. Ml-mamba: Efficient multi-modal large language model utilizing mamba-2, 2024. URL <https://arxiv.org/abs/2407.19832>. arXiv:2407.19832.
- [15] Max Jaderberg, Valentin Dalibard, Simon Osindero, et al. Population based training of neural networks. *arXiv preprint*, 2017. URL <https://arxiv.org/abs/1711.09846>. arXiv:1711.09846.
- [16] Aaron Klein, Jacek Golebiowski, Xingchen Ma, Valerio Perone, and Cédric Archambeau. Structural pruning of large language models via neural architecture search. In *AutoML Workshop*, 2023. URL <https://www.amazon.science/publications/structural-pruning-of-large-language-models-via-neural-architecture-search>.
- [17] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. Albert: A lite bert for self-supervised learning of language representations, 2019. URL <https://arxiv.org/abs/1909.11942>. arXiv:1909.11942.
- [18] Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Jonathan Tow, Alexander M. Rush, Stella Biderman, Albert Webson, Benoît Sagot, Niklas Muennighoff, Olatunji Ruwase, Rachel Bawden, Stas Bekman, Angelina McMillan-Major, et al. Bloom: A 176b-parameter open-access multilingual language model, 2022. URL <https://arxiv.org/abs/2211.05100>. arXiv:2211.05100.
- [19] Cheng Li and Yiming Yang. Multi-objective population based training. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- [20] Jun Li and Peng Zhao. Bossnas: Exploring hybrid cnn-transformers with block-wisely self-supervised neural architecture search, 2021. URL <https://arxiv.org/abs/2103.12424>. arXiv:2103.12424.
- [21] Qingyuan Li, Bo Zhang, and Xiangxiang Chu. Eapruning: Evolutionary pruning for vision transformers and cnns, 2022. URL <https://arxiv.org/abs/2210.00181>. arXiv:2210.00181.
- [22] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, et al. Jamba: A hybrid transformer-mamba language model, 2024. URL <https://arxiv.org/abs/2403.19887>. arXiv:2403.19887.

-
- [23] Han Liu, Yixuan Wang, and Yaochu Jin. Evolution meets diffusion: Neural architecture generation with diffusion models, 2025. URL <https://arxiv.org/abs/2504.17827>. arXiv:2504.17827.
 - [24] Lei Liu, Gary G. Yen, and Zhenan He. Evolutionvit: Multi-objective evolutionary vision transformer pruning under resource constraints. *Information Sciences*, 2025.
 - [25] Matyáš Lorenc and Roman Neruda. Utilizing evolution strategies to train transformers in reinforcement learning, 2025. URL <https://arxiv.org/abs/2501.13883>. arXiv:2501.13883.
 - [26] Pedro Henrique Martins, João Alves, Patrick Fernandes, Nuno M. Guerreiro, Ricardo Rei, Amin Farajian, Mateusz Klimaszewski, Duarte M. Alves, José Pombal, Nicolas Boizard, Manuel Faysse, Pierre Colombo, François Yvon, Barry Haddow, José G. C. de Souza, Alexandra Birch, and André F. T. Martins. Eurollm-9b: Technical report, 2025. URL <https://arxiv.org/abs/2506.04079>. arXiv:2506.04079.
 - [27] Haohao Qu, Liangbo Ning, Rui An, Wenqi Fan, Tyler Derr, Hui Liu, Xin Xu, and Qing Li. A survey of mamba, 2024. URL <https://arxiv.org/abs/2408.01129>. arXiv:2408.01129.
 - [28] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V. Le. Regularized evolution for image classifier architecture search. In *AAAI Conference on Artificial Intelligence*, 2019.
 - [29] Tim Salimans, Jonathan Ho, Xi Chen, Szymon Sidor, and Ilya Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. *arXiv preprint*, 2017. URL <https://arxiv.org/abs/1703.03864>. arXiv:1703.03864.
 - [30] Oliver Sieberling, Denis Kuznedelev, Eldar Kurtic, and Dan Alistarh. Evo-press: Accurate dynamic model compression via evolutionary search. In *ICML*, 2025.
 - [31] David So, Quoc Le, and Chen Liang. Evolved transformer. In *International Conference on Machine Learning (ICML)*, 2019.
 - [32] Kenneth O. Stanley and Risto Miikkulainen. Evolving neural networks through augmenting topologies. *Evolutionary Computation*, 10(2):99–127, 2002.
 - [33] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models, 2023. URL <https://arxiv.org/abs/2302.13971>. arXiv:2302.13971.
 - [34] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *NeurIPS*, 2017.
 - [35] Chao Wang, Jiaxuan Zhao, Licheng Jiao, Lingling Li, Fang Liu, and Shuyuan Yang. When large language models meet evolutionary algorithms: Potential

REFERENCES

- enhancements and challenges, 2025. URL <https://arxiv.org/abs/2401.10510>. arXiv:2401.10510.
- [36] Rui Wang, Liang Chen, and Xiang Li. Neural architecture search for transformers: A survey. *ACM Computing Surveys*, 2022.
- [37] Daan Wierstra, Tom Schaul, Jan Peters, and Jürgen Schmidhuber. Natural evolution strategies. *Journal of Machine Learning Research*, 15:949–980, 2014.
- [38] Xin Xu and Peng Zhou. Efficient gradient-free neural architecture search. *Neural Networks*, 2020.
- [39] Shangshang Yang, Xiaoshan Yu, Ye Tian, Xueming Yan, Haiping Ma, and Xingyi Zhang. Evolutionary neural architecture search for transformer in knowledge tracing. In *NeurIPS*, 2023.
- [40] Kaiyue Lu Zhen Qin Weixuan Sun Jiacheng Xu Yiran Zhong Zexiang Liu, Dong Li. Neural architecture search on efficient transformers and beyond, 2022. URL <https://arxiv.org/abs/2207.13955>. arXiv:2207.13955.
- [41] Wei Zhang, Yanan Wang, and Yaochu Jin. Multi-population evolutionary neural architecture search. *IEEE Transactions on Evolutionary Computation*, 2021.
- [42] Hao Zhao, Ming Zhang, Wei Zhao, Peng Ding, Shuai Huang, and Dong Wang. Cobra: Extending mamba to multi-modal large language model for efficient inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025. URL <https://ojs.aaai.org/index.php/AAAI/article/view/33131>.
- [43] Peng Zhou and Xin Xu. Eg-nas: Efficient neural architecture search via evolutionary graph representation. In *NeurIPS*, 2019.
- [44] Yifan Zhou, Ming Liu, and Hao Wang. Multi-objective evolutionary neural architecture search for medical image analysis using transformer and large language models in advancing public health. *Expert Systems with Applications*, 2024.